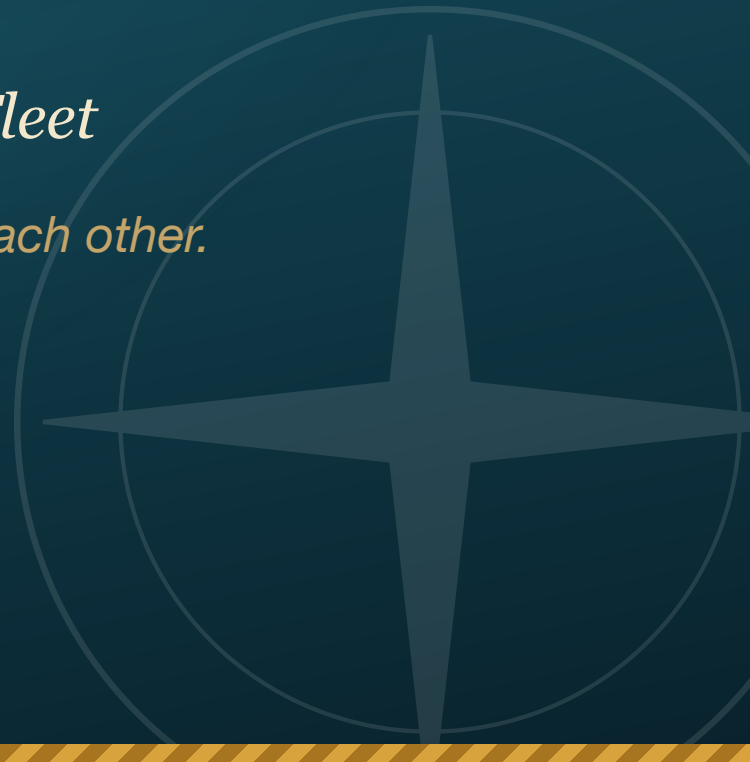
DAY 2 · LECTURE 2 · 90 MINUTES

Drawing the Fleet

Services, Internal DNS & the Three-Tier Fleet

Your Deployments are ready. Now teach them to talk to each other.

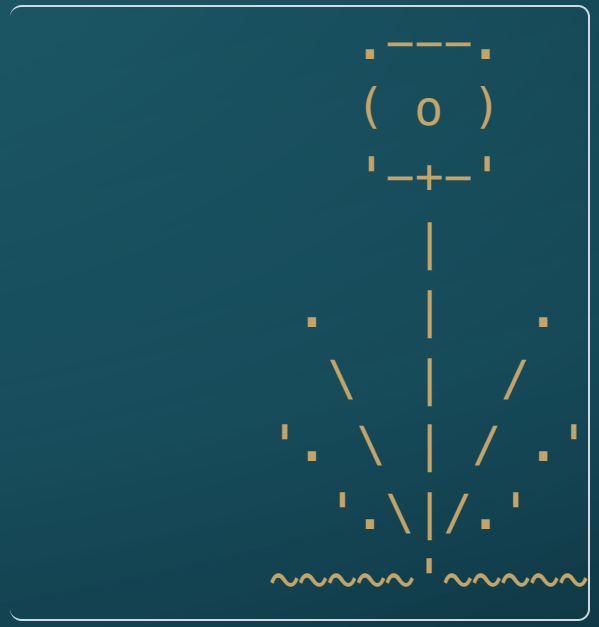


Where we are

- This morning: **resilient Deployments** — Pods die, the Deployment raises replacements.
- But here's the gap: a replacement Pod gets a **brand-new IP address** every time.
- Pod-to-Pod communication built on IPs is a ship built on sand.
- The next 90 minutes close that gap — four legs:
  - Why Pods need anchors
  - Meet the **Service** — a stable anchor for your Pods
  - Charting the sea lanes — internal DNS
  - The three-tier fleet — wiring it all together
- Then **Lab 03 — Fleet Logistics**.

PART I

Why Pods Need Anchors



> *"A crew that changes berth every tide needs a fixed notice-board — or no one finds the mess-hall." — the Boatswain*

The Ghost Ship returns

- Yesterday's "Ghost Ship": Pods are **ephemeral** by design.
- Delete a Pod, and Kubernetes raises a new one — same job, **different IP**.
- That is not a bug. It is the whole point.
- The problem: if anything else has the old IP written down, it is now pointing at **nothing**.

The IP problem, spelled out

```
Pod A                    Pod B
(backend)                (frontend)
IP: 10.0.1.15   <-- Frontend hardcodes this IP
      |
      | Pod B restarts
      v
New Pod B
IP: 10.0.1.47   <-- Frontend still dials 10.0.1.15
                    connection refused
```

- Every Pod restart hands out a fresh IP from the cluster's address pool.
- Hardcode an IP today — it is **wrong by morning**.

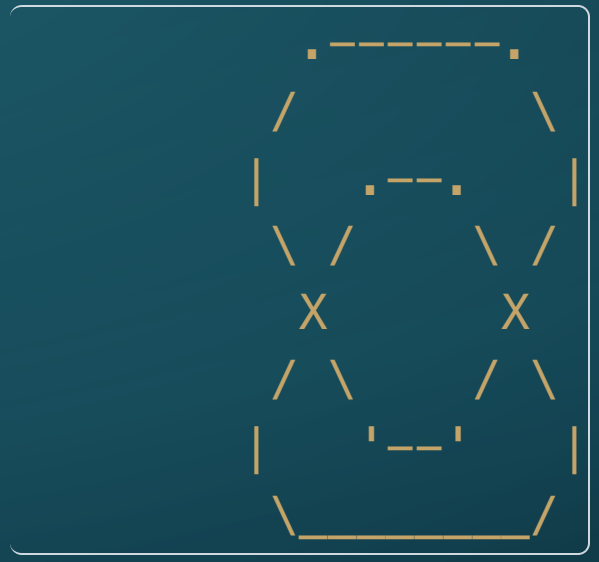
What we actually need

- A **stable name and address** that does not change when the Pod behind it does.
- Something that knows which Pods are currently healthy, and routes only to them.
- Something that **load-balances** if there are many replicas behind it.
- In short: a **fixed anchor** in front of a mobile fleet.

*That anchor has a name in Kubernetes: a **Service**.*

PART II

Meet the Service



> *"A Service don't care which hull carries the cargo — it just makes sure it gets there." — the Boatswain*

A Service is a stable anchor

- The Service holds a **fixed ClusterIP** and a **fixed DNS name**.
- Behind it, Pods come and go. The Service tracks which are healthy.
- Traffic sent to the Service is **load-balanced** across whichever Pods are ready.

How does a Service know which Pods to route to?

```
Service selector:        Pod labels:
-----------------        -----------
app: backend             app: backend    <-- matched
                         app: frontend   <-- ignored
```

- Services use **label selectors** — a set of key/value pairs.
- Any Pod whose labels **match** the selector is added to the Service's roster.
- A Pod restarts with a new IP? As long as its labels are the same, it is **automatically re-enrolled**.

Service types — three flavours

Type	Reach	Analogy
ClusterIP	Inside the cluster only	The island's internal sea lanes
NodePort	Opens a fixed port on every node	A gangplank to the outside world
LoadBalancer	Cloud-provided external IP	A real harbour with a public dock

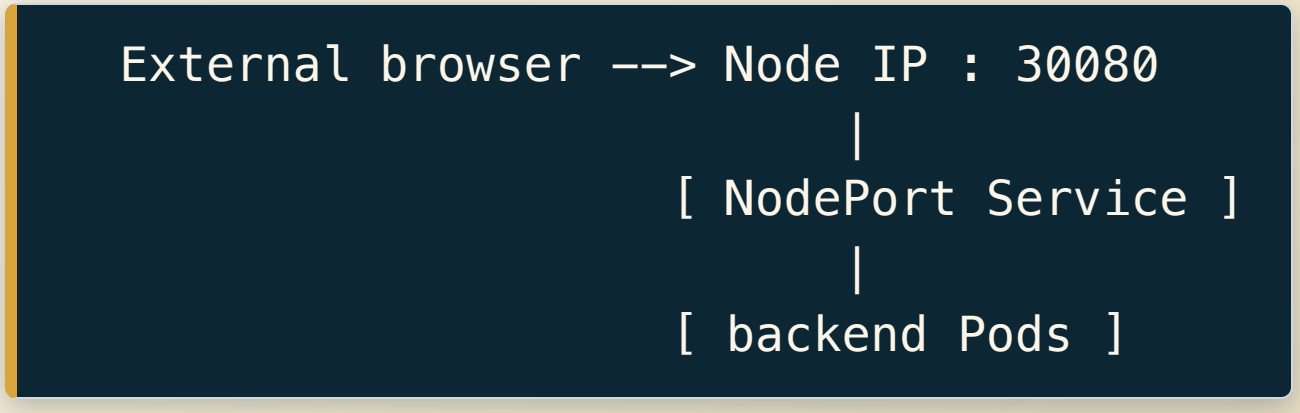
- **ClusterIP is the default** — and the one you will use most.
- NodePort and LoadBalancer are the textbook external-exposure paths. On our cluster you'll use a **Gateway HTTPRoute** in front of a ClusterIP instead (next slide).

ClusterIP — the internal sea lane

- The Service type instructors and students need most.
- Accessible **only from within the cluster** — Pods can reach it, browsers outside cannot.
- Assigned a stable IP from a reserved range; **never changes** for the life of the Service.
- Gets its own **DNS name** automatically — no IP memorisation needed.

"Internal sea lane" is the right mental model: smooth, invisible, reliable — and completely enclosed.

NodePort — the gangplank (textbook)



- Opens **one fixed port** (30000–32767) on every node in the cluster.
- Anything that can reach a node's IP can reach the Service.
- Useful for **k3d / dev clusters** where you control the node firewall.
- On our cluster: the VM's NSG blocks 30000–32767, so we don't use this path.

Gateway HTTPRoute — what we actually use

```
Browser --> radar-eric.wagbiz.org
                |
          [ main-gateway ]   (Traefik, in admin-tools NS)
                |
          [ HTTPRoute ]      (lives with your app)
                |
          [ ClusterIP Service ]   ← your normal Service
                |
          [ Pods ]
```

- The cluster runs one **Gateway** (Traefik) that owns the public IP.
- Each app owns an **HTTPRoute** that says "send hostname X to *my* ClusterIP."
- Your **Service stays ClusterIP** — internal-only. The Gateway is the front door.
- This is the production Kubernetes pattern. NodePort is the lab pattern.

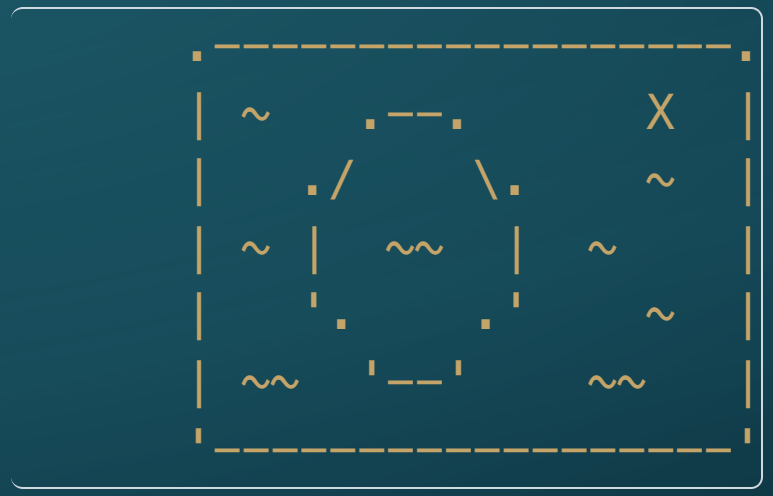
A minimal Service manifest

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: student-b
spec:
  selector:
    app: backend          # match these Pods
  ports:
    - port: 8080          # port the Service listens on
      targetPort: 8080    # port the Pod listens on
  type: ClusterIP
```

- Four moving parts: **name**, **selector**, **ports**, **type**.
- Generate the draft with `kubectl expose deployment <name> --dry-run=client -o yaml`.

PART III

Charting the Sea Lanes



> *"It is not down in any map; true places never are." — Herman Melville, Moby-Dick*

Every Service gets a DNS name

- The moment a Service is created, the cluster's **internal DNS** registers it automatically.
- No configuration needed — it is always on.
- Any Pod can resolve the Service by name.

```
curl http://backend-svc:8080
```

- Within the **same namespace**, the short name works.
- Across namespaces, you need the full form.

The full DNS name

```
<service>.<namespace>.svc.cluster.local

Example:
redis-svc.student-a.svc.cluster.local
```

- Four parts: **service name · namespace · svc · cluster.local**
- Within the **same namespace**: use just `redis-svc`
- **Across namespaces**: you must include the namespace
- Get any part wrong and the **whole logistics chain breaks**

This is the single most common mistake in the lab. Write it out, read it back.

DNS resolution — same vs. cross namespace

```
student-b namespace talking to student-a's Redis:

SAME namespace:       redis-svc                  (works if same NS)
CROSS namespace:      redis-svc.student-a        (partial — also works)
FULL form:            redis-svc.student-a.svc.cluster.local
```

- The **full form always works**. Use it for cross-namespace connections.
- Short form only works **within the same namespace**.
- Lab 03 has all three tiers in **different namespaces** — full form required.

Verifying DNS from inside a Pod

```
kubectl exec -it <pod> -- sh

# then inside:
nslookup redis-svc.student-a.svc.cluster.local
wget -qO- http://backend-svc.student-b.svc.cluster.local:8080
```

- `nslookup` tells you if the name resolves at all.
- `wget` or `curl` tells you if the Service responds.
- If `nslookup` fails: the Service name or namespace is wrong.
- If `nslookup` works but `curl` fails: the port or selector is wrong.

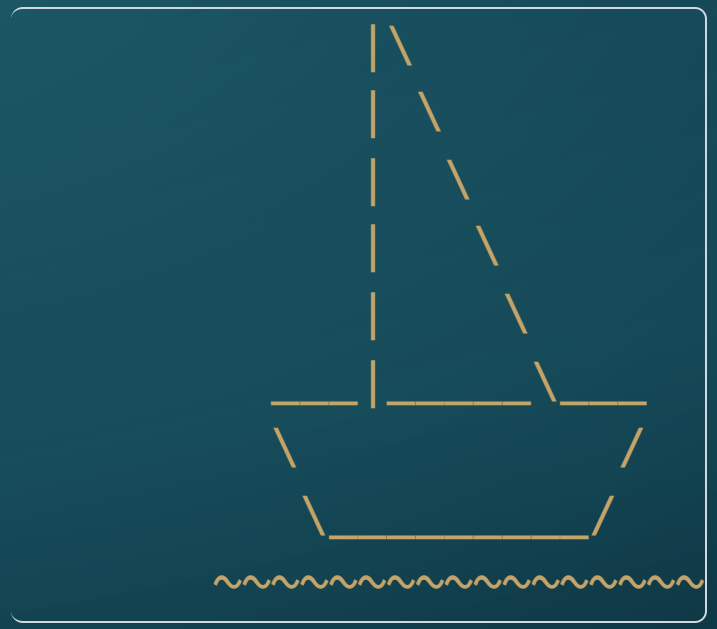
The DNS contract

> **Rule:** *If it isn't a Service DNS name, don't put it in an environment variable.*

- No hardcoded Pod IPs — they are wrong by morning.
- No node IPs — unreliable across restarts.
- **Only Service DNS names** travel between tiers.
- The AI Code Reviewer in your `AGENTS.md` will flag violations automatically.

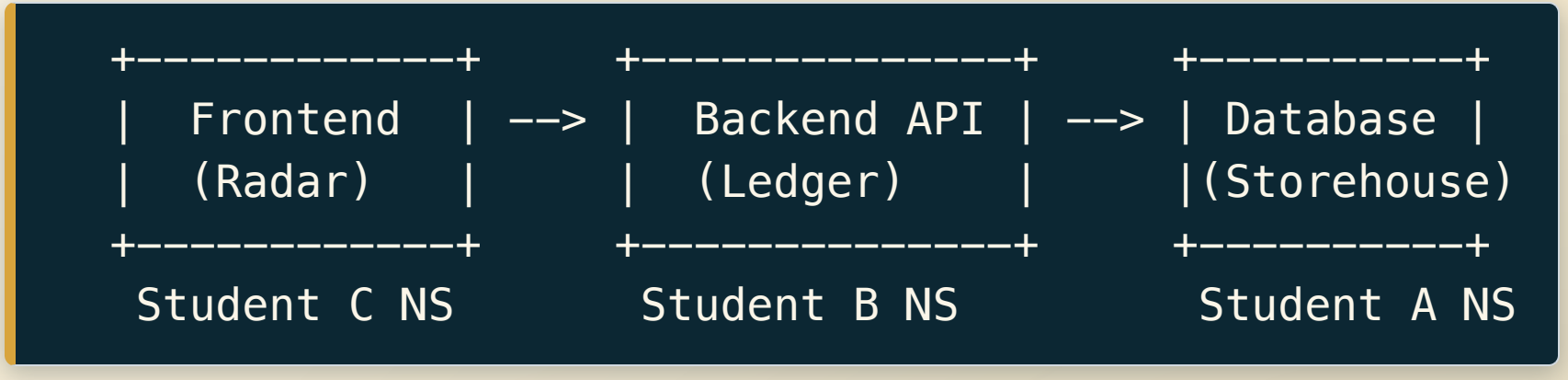
PART IV

The Three-Tier Fleet



> *"Three ships in formation beat one ship alone — but only if the signals are right." — the Boatswain*

The classic three-tier application



- Each tier is an independent **Deployment + Service**.
- Each student owns one tier in their own **namespace**.
- The tiers communicate **only through Service DNS names** — no direct Pod access.

Wiring the tiers — environment variables

- **Student A (Storehouse)**: deploys Redis, creates a ClusterIP Service.
  - No outbound dependency — the Storehouse receives, it doesn't call.
- **Student B (Ledger)**: deploys the backend API, sets `CACHE_URL`:

```
tcp://redis-svc.student-a.svc.cluster.local:6379
```

- **Student C (Radar)**: deploys the frontend, sets `BACKEND_URL`:

```
http://backend-svc.student-b.svc.cluster.local:8080
```

The dependency chain

```
Student C's Frontend
        |
        | BACKEND_URL = backend-svc.student-b.svc.cluster.local
        v
Student B's Backend API
        |
        | CACHE_URL = redis-svc.student-a.svc.cluster.local
        v
Student A's Redis
```

- If Student A's Service name is wrong, **Student B's link to it silently dies** — the pod stays green.
- If Student B's Service name is wrong, **Student C's link silently dies** the same way.
- Nothing crashes. The break only surfaces when you *test* the connection from inside the dependent Pod.

What each student writes

Student	Tier	Image	Exposed By
A	Redis cache	`redis:7.2-alpine`	ClusterIP
B	Go API	`stefanprodan/podinfo:6.7.0`	ClusterIP
C	Web UI	`paulbouwer/hello-kubernetes:1.10`	ClusterIP + HTTPRoute

- Pre-built images are provided — focus stays on **YAML, not application code**.
- Every student writes exactly: **one Deployment + one Service**.
- The AI Code Reviewer critiques the YAML before it is applied.

If the namespace is wrong

```
Student B sets:
CACHE_URL = tcp://redis-svc.student-c.svc.cluster.local:6379
                                    ^
                                    wrong namespace

Result: Pod B stays GREEN — it never dials Redis on boot
        The name points at the wrong namespace (or nowhere)
        The break is invisible until someone tests the link
```

- The **link** is broken. The **pods** are fine.
- `kubectl get pods` shows green across the board.
- Only `kubectl exec` + `nslookup` reveals the actual fault.

A word on NetworkPolicy

- A **NetworkPolicy** is a cluster-level firewall rule.
- A **default-deny** policy blocks all cross-namespace traffic — even between healthy Pods and correct Services.
- This is the lab's **Network Blockade** twist: apps break even though nothing is crashed.
- The fix is writing allow rules that name the permitted source namespace.

We will say no more about it here. You will know it when you meet it.

Instructor Superpower

THE COLLABORATIVE CLASSROOM

- Services and internal DNS let you wire **real dependencies between students**.
- Student A owns the database. Student B owns the backend. Student C owns the frontend — each in their own namespace.
- This mimics real **cross-team microservice development**: if A's Service is misconfigured, C's tier can't reach it down the chain.
- Students debug an authentic distributed system — not a toy exercise.
- You can stand this up yourself — for one class session or a whole semester — with no new infrastructure to request.

What this classroom can do

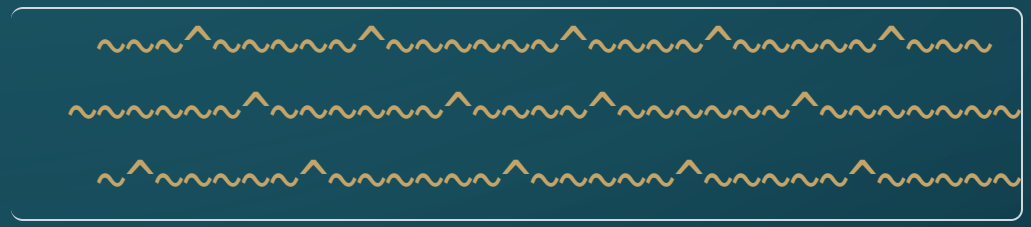
- **Accountability is visible**: if Student A's Service is down, Students B and C know it.
- **Debugging is collaborative**: `kubectl exec` + DNS tools, across the team.
- **Failure is meaningful**: a wrong namespace in one YAML breaks the whole chain — not as punishment, but as an accurate model of how distributed systems actually fail.
- **Recovery is theirs to own**: the instructor does not fix it; the team does.

Up next: Lab 03 — Fleet Logistics

- Form **3-person alliances**: one student per tier.
- Each student writes **one Deployment + one Service** for their tier.
- Wire the tiers together using **internal DNS names** across namespaces.
- Use the AI Code Reviewer to catch IP-hardcoding before it sinks the chain.
- If the fleet comes up and runs — well done. The ocean isn't done with you yet.

> *Open `lab-03-fleet-logistics.md` on the docs site. Know your teammates' namespaces before you touch any YAML.*

Man the sea lanes.



The fleet is drawn. Wire it together.